WebSockets

Connection URLs, authentication, event fields, message processing, and reconnect behavior.

Overview

Caisarra uses OkHttp WebSocket clients for live online human games and bot games. REST is used for creation, history, profiles, chat history, and related data; WebSocket is used for live game events and low-latency actions.

Transport setup

Source: data/remote/RetrofitInstance.kt

```
RetrofitInstance.socketClient = OkHttpClient.Builder() .pingInterval(20, TimeUnit.SECONDS) .build()
```

Human game connection

Source: data/remote/GameSocketClient.kt

```
https://caisaara.duckdns.org/ws/{gameId}?token={accessToken}
```

Bot game connection

Source: data/remote/BotSocketClient.kt

```
https://caisaara.duckdns.org/ws/bot-games/{gameId}?token={accessToken}
```

Connection sequence

```
1. close() previous socket and increment generation. 2. AuthRepository.isLoggedIn() validates the token and refreshes if expired. 3. TokenManager.getAccessToken() reads the current token. 4. Build the WebSocket URL with the game ID and token query parameter. 5. OkHttp newWebSocket() registers a WebSocketListener. 6. onOpen sets connected=true. 7. onMessage decodes JSON and emits typed incoming events. 8. onClosing/onClosed/onFailure update connected=false. 9. OnlineGameViewModel can reconnect after an unexpected disconnect.
```

Human outgoing protocol

Source: data/remote/dto/SocketOutgoing.kt

Type	Fields	Purpose
move	move: String	Submit a chess move.
chat	message: String	Send an in-game chat message.
resign	none	Resign the game.
offer_draw	none	Offer a draw.
accept_draw	none	Accept a draw.
decline_draw	none	Decline a draw.

Human incoming protocol

Source: data/remote/dto/SocketIncoming.kt

Type	Fields	Purpose
game_start	game_id: String	Game has started.
game_state	game_id, position, status, white_time_ms, black_time_ms, current_turn, turn_started_at, white_player_id, black_player_id, white_username, black_username, last_move, moves, result, end_reason, draw_offered_by, time_control_minutes	Full synchronized game state.
waiting	game_id: String	Waiting for opponent.
draw_offer	offered_by: Long	Opponent offered a draw.
draw_declined	declined_by: Long	Draw was declined.
error	message: String	Server error.
chat	id, game_id, sender_id, message, created_at	Incoming chat message.

Bot protocol

Source: data/remote/dto/BotSocketOutgoing.kt and BotSocketIncoming.kt

Direction	Type	Fields
outgoing	move	move: String
outgoing	resign	none
incoming	game_state	game: BotGameDto
incoming	bot_thinking	none
incoming	error	message: String

Message processing

GameSocketClient normalizes server messages that use event instead of type. It decodes the normalized JSON with kotlinx.serialization and emits SocketIncoming through a MutableSharedFlow. A game_start event also causes a synthesized game_state event so the ViewModel can initialize state consistently.

ViewModel integration

```
OnlineGameViewModel socket.incoming.collect { handle(it) } socket.connected.collect { reconnect when unexpected disconnect } socket.connect(gameId) GameSocketClient.send(SocketOutgoing.Move(...))
BotGameViewModel BotSocketClient.incoming.collect { handle(...) } BotSocketClient.connect(gameId)
BotSocketClient.send(BotSocketOutgoing.Move(...))
```

Refresh and reconnect behavior

REST requests use AuthInterceptor to refresh an expiring access token and TokenAuthenticator to retry a 401 once. WebSockets do not use Retrofit interceptors, so both socket clients explicitly call AuthRepository.isLoggedIn() before connecting, then read the refreshed token from TokenManager. If an established socket disconnects, OnlineGameViewModel attempts reconnection.

Important boundary

A WebSocket connection is not automatically refreshed while it is already open. If the token expires during a long-running socket session, the current implementation relies on reconnecting to obtain a fresh token. The server must accept the token query parameter format used by these clients.